

TECHNICAL TRAINING COURSE

Content and Lifecycle in JEE

JAX-RS 2.x | Resources, JNDI, Databases, and Security



Topics Covered: Reading Web Resources · Finding Java EE Components via JNDI
Finding Databases via DataSource · Security Contexts

Platform: Java 11 LTS | Tomcat 9.0 | JAX-RS 2.1

Namespace: javax.ws.rs.* / javax.servlet.* (NOT jakarta.*)

Duration: 1 Day (8 hours)

Level: Intermediate Java Developer

Table of Contents

Table of Contents	2
1. Reading Web Resources	4
1.1 Learning Objectives	4
1.2 The Web Resource Model	4
1.3 Injecting ServletContext with @Context	4
1.4 Reading Files from the WAR	5
1.5 Context Init Parameters from web.xml.....	5
1.6 Injecting HttpServletRequest and HttpServletResponse	6
1.7 Application Lifecycle Events — ServletContextListener	6
1.8 Review Questions	7
2. Finding Java EE Components	8
2.1 Learning Objectives	8
2.2 JNDI — The JEE Naming Registry	8
2.3 Programmatic JNDI Lookup.....	8
2.4 Registering Resources in Tomcat context.xml	9
2.5 Binding Custom Objects to JNDI at Startup.....	9
2.6 @Resource Injection	10
2.7 Review Questions	10
3. Finding Databases	11
3.1 Learning Objectives	11
3.2 DataSource and Connection Pooling.....	11
3.3 JNDI DataSource Lookup in a JAX-RS Resource	11
3.4 Storing the DataSource in ServletContext	12
3.5 Correct try-with-resources Usage	13
3.6 Transaction Boundaries in Tomcat	13
3.7 Review Questions	14
4. Security Contexts	15
4.1 Learning Objectives	15
4.2 Authentication vs Authorisation	15
4.3 The SecurityContext Interface	15
4.4 Implementing a Bearer Token Authentication Filter.....	16
4.5 Custom SecurityContext Implementation	16
4.6 Declarative Authorisation — @RolesAllowed.....	17
4.7 @Secured Name-Binding for Selective Authentication	18
4.8 Complete Security Flow	19
4.9 Review Questions	19
Appendix — Quick Reference	20

javax.* vs jakarta.* — Tomcat 9 Package Guide	20
Response Format Summary	20

1. Reading Web Resources

1.1 Learning Objectives

- Inject servlet context resources into JAX-RS components using `@Context`
- Read static files from the WAR using `ServletContext.getResourceAsStream()`
- Understand the difference between classpath resources and web context resources
- Read application configuration from `web.xml` init parameters
- Use `@Context` `HttpServletRequest` to access raw servlet facilities

1.2 The Web Resource Model

A deployed WAR contains two types of resources that a JAX-RS resource class can read at runtime: web-context resources (files under the WAR root, accessible via `ServletContext`) and classpath resources (files under `WEB-INF/classes` or inside JARs, accessible via `ClassLoader.getResourceAsStream()`).

Term	Definition
Web context resource	A file physically under the WAR root. Examples: <code>/WEB-INF/config.xml</code> , <code>/static/template.html</code> . Accessed via <code>ServletContext.getResourceAsStream(path)</code> .
Classpath resource	A file on the Java classpath — typically in <code>src/main/resources</code> (packaged to <code>WEB-INF/classes</code>). Accessed via <code>getClass().getResourceAsStream("/path")</code> .
ServletContext	The JEE object representing the deployed web application. Injected via <code>@Context</code> in JAX-RS. Provides access to init-params, JNDI, and real paths.
@Context	JAX-RS annotation that injects framework and container objects into resource classes, providers, and Application subclasses.

1.3 Injecting ServletContext with @Context

`ServletContext` can be injected directly into a JAX-RS resource class field or method. Import from `javax.servlet` (NOT `jakarta.servlet`) for Tomcat 9:

Package Namespace & Response Format

All code uses `javax.ws.rs.*` and `javax.servlet.*` (NOT `jakarta.*`) — correct for Java 11 / Tomcat 9.

`javax.annotation.security.RolesAllowed` / `@PermitAll` / `@DenyAll` (NOT `jakarta.annotation.security.*`)

Response format strategy:

GET `/health` → `APPLICATION_XML` (`@XmlElement` JAXB bean)

Section 1 & 2 (config, JNDI) → `TEXT_PLAIN` (raw strings / `key=value` lines)

Section 3 (JDBC results) → `TEXT_PLAIN` (CSV rows — tabular data, not domain objects)

Section 4 (security errors) → `TEXT_PLAIN` (`error=...` lines; HTTP status carries meaning)

```
import javax.ws.rs.*;
import javax.ws.rs.core.*;
import javax.servlet.ServletContext;

@Path("/config")
public class ConfigResource {

    // Field injection - works in JAX-RS resource classes
    @Context
    private ServletContext servletContext;

    @GET @Path("version")
    @Produces(MediaType.TEXT_PLAIN)
    public String version() {
        // Read a context init-param from web.xml
        String version = servletContext.getInitParameter("app.version");
        return version != null ? version : "unknown";
    }

    @GET @Path("name")
    @Produces(MediaType.TEXT_PLAIN)
    public String appName() {
        return servletContext.getServletContextName();
    }
}
```

1.4 Reading Files from the WAR

`ServletContext.getResourceAsStream(path)` reads a file relative to the WAR root. The path must start with `/`. This is the correct way to read configuration files, templates, and schemas bundled inside the WAR:

```
@GET @Path("template")
@Produces(MediaType.TEXT_HTML)
public Response getTemplate() throws IOException {
    // Reads /WEB-INF/templates/welcome.html from the WAR root
    InputStream is = servletContext
        .getResourceAsStream("/WEB-INF/templates/welcome.html");
    if (is == null) return Response.status(404).build();
    String html = new String(is.readAllBytes(), StandardCharsets.UTF_8);
    return Response.ok(html).build();
}

// Compare with classpath resource (from WEB-INF/classes):
InputStream is =
    getClass().getResourceAsStream("/config/defaults.properties");
// In Maven, this file lives at src/main/resources/config/defaults.properties
```

1.5 Context Init Parameters from web.xml

Application-wide configuration can be declared in `web.xml` as context-param elements. Use the Servlet 4.0 namespace for Tomcat 9:

```
<!-- web.xml - Servlet 4.0 / Tomcat 9 -->
<web-app xmlns="http://xmlns.jcp.org/xml/ns/javaee"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
        http://xmlns.jcp.org/xml/ns/javaee/web-app_4_0.xsd"
```

```

        version="4.0">
<display-name>My API</display-name>
<context-param>
    <param-name>app.version</param-name>
    <param-value>2.4.1</param-value>
</context-param>
<context-param>
    <param-name>feature.payments.enabled</param-name>
    <param-value>true</param-value>
</context-param>
<context-param>
    <param-name>upload.max.bytes</param-name>
    <param-value>10485760</param-value>
</context-param>
</web-app>

// Reading init-params in Java:
String version = ctx.getInitParameter("app.version");           // "2.4.1"
boolean payments = Boolean.parseBoolean(
    ctx.getInitParameter("feature.payments.enabled"));          // true
long maxBytes = Long.parseLong(
    ctx.getInitParameter("upload.max.bytes"));                  // 10485760

```

1.6 Injecting HttpServletRequest and HttpServletResponse

The raw Servlet request and response objects are also injectable with `@Context`. Import from `javax.servlet.http` (NOT `jakarta.servlet.http`):

```

import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.servlet.http.HttpSession;

@GET @Path("client-info")
@Produces(MediaType.TEXT_PLAIN)
public Response clientInfo(@Context HttpServletRequest request) {
    String info = "remoteAddr=" + request.getRemoteAddr() + "\n" +
        "remoteHost=" + request.getRemoteHost() + "\n" +
        "localPort=" + request.getLocalPort() + "\n" +
        "scheme=" + request.getScheme() + "\n" +
        "sessionId=" + getOrCreateSessionId(request) + "\n";
    return Response.ok(info).build();
}

private String getOrCreateSessionId(HttpServletRequest req) {
    HttpSession session = req.getSession(false); // false = don't create
    return session != null ? session.getId() : "(no session)";
}

```

1.7 Application Lifecycle Events — ServletContextListener

For one-time startup initialisation, implement a `@WebListener`. Import from `javax.servlet` (NOT `jakarta.servlet`):

```

import javax.servlet.ServletContext;
import javax.servlet.ServletContextEvent;
import javax.servlet.ServletContextListener;
import javax.servlet.annotation.WebListener;

```

```
@WebListener
public class AppLifecycleListener implements ServletContextListener {

    @Override
    public void contextInitialized(ServletContextEvent sce) {
        ServletContext ctx = sce.getServletContext();
        String version = ctx.getInitParameter("app.version");
        ctx.log("Application starting - version: " + version);
        // Load a WAR resource at startup
        InputStream is = ctx.getResourceAsStream("/WEB-INF/config/schema.xsd");
        // ... compile and store in ServletContext attribute
        ctx.setAttribute("schema", compileSchema(is));
        ctx.log("Application initialised successfully.");
    }

    @Override
    public void contextDestroyed(ServletContextEvent sce) {
        sce.getServletContext().log("Application shutting down.");
    }
}
```

1.8 Review Questions

1. Explain the difference between `getResourceAsStream("/WEB-INF/data.xml")` on a `ServletContext` and `getClass().getResourceAsStream("/data.xml")` on a `ClassLoader`.
2. A `web.xml` `context-param` has `param-name` `app.env` and `param-value` `production`. Write the Java code to read this value in a JAX-RS resource class using `javax.servlet.ServletContext`.
3. Why should you use `@Context HttpServletRequest.getSession(false)` rather than `getSession()` in an API resource?

2. Finding Java EE Components

2.1 Learning Objectives

- Understand JNDI as the naming and directory service for locating JEE components
- Perform JNDI lookups for DataSources and arbitrary named resources
- Bind resources to JNDI from a ServletContextListener
- Use @Resource injection as an alternative to programmatic JNDI lookup
- Handle JNDI naming exceptions correctly

2.2 JNDI — The JEE Naming Registry

Java Naming and Directory Interface (JNDI) is the standard mechanism for locating named JEE resources at runtime. In Tomcat 9 (a servlet container), you configure resources explicitly in context.xml.

Term	Definition
InitialContext	The entry point for all JNDI lookups. Uses the default server naming service.
java:comp/env	The 'component environment' naming context. All application-scoped resources are looked up relative to this prefix.
java:comp/env/jdbc/	Conventional prefix for JDBC DataSources in Tomcat context.xml.
java:comp/env/jms/	Conventional prefix for JMS connection factories and destinations.
NameNotFoundException	Thrown when the requested name does not exist in the JNDI registry.

2.3 Programmatic JNDI Lookup

```
import javax.naming.*;
import javax.sql.DataSource;

// Generic pattern for any JNDI-registered component
public <T> T jndiLookup(String jndiName, Class<T> type) {
    try {
        Context ctx = new InitialContext();
        Object resource = ctx.lookup(jndiName);
        return type.cast(resource);
    } catch (NameNotFoundException e) {
        throw new IllegalStateException(
            "JNDI resource not found: " + jndiName, e);
    } catch (NamingException e) {
        throw new IllegalStateException(
            "JNDI lookup failed: " + jndiName, e);
    }
}

// Example usages:
DataSource ds = jndiLookup(
```

```
"java:comp/env/jdbc/shopDB", DataSource.class);
```

2.4 Registering Resources in Tomcat context.xml

```
<!-- src/main/webapp/META-INF/context.xml -->
<Context>
  <!-- JDBC DataSource -->
  <Resource name="jdbc/shopDB"
    auth="Container"
    type="javax.sql.DataSource"
    driverClassName="org.h2.Driver"
    url="jdbc:h2:mem:shopdb;DB_CLOSE_DELAY=-1;MODE=PostgreSQL"
    username="sa" password=""
    maxTotal="10" maxIdle="5" maxWaitMillis="-1"/>

  <!-- Custom string parameter -->
  <Environment name="app.version"
    type="java.lang.String"
    value="2.4.1"
    override="false"/>
</Context>

// Corresponding web.xml resource-ref:
// <resource-ref>
//   <res-ref-name>jdbc/shopDB</res-ref-name>
//   <res-type>javax.sql.DataSource</res-type>
//   <res-auth>Container</res-auth>
// </resource-ref>
```

2.5 Binding Custom Objects to JNDI at Startup

```
import javax.servlet.*;
import javax.servlet.annotation.WebListener;
import javax.naming.*;

@WebListener
public class ResourceBindingListener implements ServletContextListener {

  @Override
  public void contextInitialized(ServletContextEvent sce) {
    try {
      InitialContext ic = new InitialContext();
      // Ensure the subcontext exists
      try { ic.lookup("java:comp/env/app"); }
      catch (NameNotFoundException e) {
        ic.createSubcontext("java:comp/env/app"); }
      // Bind a compiled XML Schema
      Schema schema = compileOrderSchema();
      ic.rebind("java:comp/env/app/orderSchema", schema);
      sce.getServletContext().log("Resources bound to JNDI.");
    } catch (NamingException e) {
      throw new RuntimeException("JNDI binding failed", e);
    }
  }

  @Override
  public void contextDestroyed(ServletContextEvent sce) {
    try { new InitialContext().unbind("java:comp/env/app"); }
  }
}
```

```
        catch (NamingException ignored) {}
    }
}
```

2.6 @Resource Injection

In Tomcat 9, @Resource injection works for DataSources and environment entries configured in context.xml. Import from javax.annotation (NOT jakarta.annotation):

```
import javax.annotation.Resource;
import javax.sql.DataSource;

// In a Servlet (Tomcat supports @Resource for DataSource and env entries):
@Resource(name = "jdbc/shopDB")
private DataSource dataSource;

@Resource(name = "app.version")
private String appVersion;

// JAX-RS resource classes do NOT support @Resource directly in Tomcat.
// Use @Context ServletContext + programmatic JNDI lookup instead:
@Path("/orders")
public class OrderResource {

    @Context
    private javax.servlet.ServletContext servletContext;

    private DataSource getDataSource() {
        try {
            Context ctx = new InitialContext();
            return (DataSource) ctx.lookup("java:comp/env/jdbc/shopDB");
        } catch (NamingException e) {
            throw new IllegalStateException("DataSource not found", e);
        }
    }
}
```

2.7 Review Questions

4. What is the conventional JNDI name prefix for a JDBC DataSource in Tomcat 9?
5. Explain why JNDI lookups inside a JAX-RS resource method should always look up a DataSource rather than holding a Connection as a field.
6. A @WebListener binds a resource to java:comp/env/app/schema at startup. Write the corresponding JNDI lookup in a resource class.

3. Finding Databases

3.1 Learning Objectives

- Locate a DataSource via JNDI and open connections from a resource class
- Use try-with-resources correctly for Connection, PreparedStatement, and ResultSet
- Understand connection pool behaviour and why connections must always be returned
- Access a DataSource stored in a ServletContext attribute as an alternative to JNDI
- Understand transaction boundaries in a non-JTA Tomcat environment

3.2 DataSource and Connection Pooling

Term	Definition
DataSource	Interface (javax.sql) that abstracts connection acquisition. Obtained via JNDI or direct instantiation.
Connection pool	A set of pre-opened database connections held by the DataSource. Borrowing and returning is fast; creating a new physical connection is slow.
ds.getConnection()	Borrows a connection from the pool. Must always be closed (returned) — even on exception.
connection.close()	Returns the connection to the pool (does NOT physically close the socket in a pooled scenario).
maxTotal	Maximum connections the pool will open. Calls block (or throw) when the pool is exhausted.
maxWaitMillis	How long a borrower waits for a connection before throwing. -1 = wait forever (dangerous in production).

3.3 JNDI DataSource Lookup in a JAX-RS Resource

The JDBC results in the list endpoint are plain tabular data — not domain objects — so they are returned as TEXT_PLAIN CSV rows rather than XML:

Why TEXT_PLAIN for JDBC results?

JDBC queries return raw column values (strings, numbers, timestamps) from an arbitrary SQL query — these are not JAX-RS domain model objects annotated with @XmlRootElement. Returning them as TEXT_PLAIN CSV rows is honest and avoids forcing a JAXB wrapper class on ad-hoc diagnostic or reporting queries.

For structured domain data (e.g. a Product entity read from DB), annotate the domain class with @XmlRootElement and return it with APPLICATION_XML.

```
import javax.servlet.ServletContext;
import javax.ws.rs.*;
import javax.ws.rs.core.*;
import javax.naming.*;
import javax.sql.DataSource;
```

```

import java.sql.*;

@Path("/products")
public class ProductResource {

    private DataSource dataSource;

    @Context
    public void setServletContext(ServletContext ctx) {
        try {
            Context ic = new InitialContext();
            this.dataSource = (DataSource) ic.lookup(
                "java:comp/env/jdbc/shopDB");
        } catch (NamingException e) {
            throw new RuntimeException("DataSource not found", e);
        }
    }

    // Returns CSV rows – plain tabular data, not a domain object
    @GET
    @Produces(MediaType.TEXT_PLAIN)
    public String list() {
        StringBuilder sb = new StringBuilder("id,name,category,price\n");
        String sql = "SELECT id, name, category, price FROM products ORDER BY
id";
        try (Connection conn = dataSource.getConnection();
            PreparedStatement ps = conn.prepareStatement(sql);
            ResultSet rs = ps.executeQuery()) {
            while (rs.next()) {
                sb.append(rs.getString("id")).append(",")
                  .append(rs.getString("name")).append(",")
                  .append(rs.getString("category")).append(",")
                  .append(rs.getBigDecimal("price")).append("\n");
            }
        } catch (SQLException e) {
            throw new InternalServerErrorException(
                "Database error: " + e.getMessage(), e);
        }
        return sb.toString();
    }
}

```

3.4 Storing the DataSource in ServletContext

```

import javax.servlet.*;
import javax.servlet.annotation.WebListener;
import javax.naming.*;
import javax.sql.DataSource;

@WebListener
public class DatabaseListener implements ServletContextListener {
    public static final String DS_ATTR = "javax.sql.DataSource";

    @Override
    public void contextInitialized(ServletContextEvent sce) {
        try {
            Context ic = new InitialContext();
            DataSource ds = (DataSource) ic.lookup(
                "java:comp/env/jdbc/shopDB");
            sce.getServletContext().setAttribute(DS_ATTR, ds);
            sce.getServletContext().log("DataSource bound to context.");
        }
    }
}

```

```

        } catch (NamingException e) {
            throw new RuntimeException("DataSource lookup failed", e);
        }
    }
}

// In a JAX-RS resource class:
@Context
private javax.servlet.ServletContext servletContext;

private DataSource getDs() {
    return (DataSource)
        servletContext.getAttribute(DatabaseListener.DS_ATTR);
}

```

3.5 Correct try-with-resources Usage

```

// CORRECT — all three resources closed in reverse order, even on exception
try (Connection conn = ds.getConnection();
    PreparedStatement ps = conn.prepareStatement(
        "SELECT * FROM orders WHERE customer_id = ?");
    ResultSet rs = execute(ps, customerId)) {
    while (rs.next()) {
        // process row
    }
} catch (SQLException e) {
    throw new InternalServerErrorException(e.getMessage(), e);
}

// WRONG — connection leaks if an exception occurs after open
Connection conn = ds.getConnection(); // LEAKED if next line throws
PreparedStatement ps = conn.prepareStatement(...);
ResultSet rs = ps.executeQuery();
// ... process ...
rs.close(); ps.close(); conn.close(); // never reached on exception

```

3.6 Transaction Boundaries in Tomcat

Tomcat 9 does not provide JTA by default. Database transactions must be managed manually. The POST body is TEXT_PLAIN (key=value format) to stay consistent with the module's format strategy:

```

// POST /api/db/products
// Body: id=PNEW,name=Design Patterns,category=books,price=44.99
@POST
@Consumes(MediaType.TEXT_PLAIN)
@Produces(MediaType.TEXT_PLAIN)
public Response createProduct(String body) {
    // Parse key=value pairs from the plain-text body
    Map<String,String> params = new LinkedHashMap<>();
    for (String pair : body.trim().split(",")) {
        String[] kv = pair.split("=", 2);
        if (kv.length == 2) params.put(kv[0].trim(), kv[1].trim());
    }
    String id      = params.get("id");
    String name    = params.get("name");
    String category = params.get("category");
    double price   = Double.parseDouble(params.getOrDefault("price", "0"));
}

```

```

try (Connection conn = ds.getConnection()) {
    conn.setAutoCommit(false); // begin transaction
    try {
        try (PreparedStatement ps = conn.prepareStatement(
            "INSERT INTO products(id,name,category,price)
VALUES(?,?,?,?)")) {
            ps.setString(1,id); ps.setString(2,name);
            ps.setString(3,category); ps.setDouble(4,price);
            ps.executeUpdate();
        }
        try (PreparedStatement ps = conn.prepareStatement(
            "INSERT INTO audit_log(action,entity_id) VALUES(?,?)")) {
            ps.setString(1,"CREATE"); ps.setString(2,id);
            ps.executeUpdate();
        }
        conn.commit(); // all-or-nothing
        return Response.status(201)
            .entity("created=" + id + "\n").build();
    } catch (Exception e) {
        conn.rollback(); // undo all changes
        throw new InternalServerErrorException(e.getMessage(), e);
    }
} catch (SQLException e) {
    throw new InternalServerErrorException(e.getMessage(), e);
}
}

```

3.7 Review Questions

7. Why should a JAX-RS resource class cache the DataSource reference but NOT cache a Connection?
8. A connection pool has maxTotal=5. Six concurrent requests arrive simultaneously. What happens to the sixth request?
9. Write the correct try-with-resources block to execute a parameterised UPDATE statement and return the number of affected rows.

4. Security Contexts

4.1 Learning Objectives

- Use `@Context SecurityContext` to check authentication and authorisation in resource methods
- Apply declarative security with `@RolesAllowed`, `@PermitAll`, and `@DenyAll`
- Implement a `ContainerRequestFilter` for Bearer token authentication
- Understand the difference between authentication and authorisation
- Return 401 Unauthorized and 403 Forbidden correctly

4.2 Authentication vs Authorisation

Term	Definition
Authentication	Verifying who the caller is. Example: validating a Bearer token or username+password against a database.
Authorisation	Verifying that the authenticated caller has permission to perform the requested action. Example: checking that the user has the 'admin' role.
401 Unauthorized	The caller has not authenticated (or credentials are invalid). Response should include WWW-Authenticate.
403 Forbidden	The caller is authenticated but does not have the required role or permission.
SecurityContext	JAX-RS interface injected via <code>@Context</code> . Provides: <code>getUserPrincipal()</code> , <code>isUserInRole(String)</code> , <code>isSecure()</code> , <code>getAuthenticationScheme()</code> .

4.3 The SecurityContext Interface

`SecurityContext` is injected by the JAX-RS runtime after authentication is complete. Import from `javax.ws.rs.core` (NOT `jakarta.ws.rs.core`):

```
import javax.ws.rs.core.SecurityContext;
import javax.ws.rs.*;
import javax.ws.rs.core.*;

@GET @Path("profile")
@Produces(MediaType.TEXT_PLAIN)
public Response getProfile(@Context SecurityContext security) {
    if (!security.isSecure()) {
        return Response.status(Response.Status.FORBIDDEN)
            .entity("error=HTTPS required\n").build();
    }
    java.security.Principal principal = security.getUserPrincipal();
    if (principal == null) {
        return Response.status(Response.Status.UNAUTHORIZED)
            .header("WWW-Authenticate", "Bearer realm=\"api\"")
            .entity("error=Authentication required\n").build();
    }
    if (!security.isUserInRole("user")) {
        return Response.status(Response.Status.FORBIDDEN)
            .entity("error=Forbidden\n").build();
    }
    // ... rest of the method
}
```

```

        .entity("error=Insufficient permissions\n").build();
    }
    return Response.ok(
        "username=" + principal.getName() + "\n"
    ).build();
}

```

4.4 Implementing a Bearer Token Authentication Filter

Import all filter types from javax.ws.rs.* and javax.annotation.* (NOT jakarta.*). Error responses are TEXT_PLAIN — the HTTP status code carries the semantic meaning:

```

import javax.annotation.Priority;
import javax.ws.rs.Priorities;
import javax.ws.rs.container.ContainerRequestContext;
import javax.ws.rs.container.ContainerRequestFilter;
import javax.ws.rs.core.*;
import javax.ws.rs.ext.Provider;

@Provider
@Priority(Priorities.AUTHENTICATION) // runs before resource methods
public class BearerTokenFilter implements ContainerRequestFilter {

    private static final String BEARER_PREFIX = "Bearer ";

    @Override
    public void filter(ContainerRequestContext ctx) {
        String authHeader = ctx.getHeaderString(HttpHeaders.AUTHORIZATION);
        if (authHeader == null || !authHeader.startsWith(BEARER_PREFIX)) {
            abort(ctx, Response.Status.UNAUTHORIZED,
                "Missing or malformed Authorization header");
            return;
        }
        String token = authHeader.substring(BEARER_PREFIX.length()).trim();
        UserPrincipal principal = validateToken(token);
        if (principal == null) {
            abort(ctx, Response.Status.UNAUTHORIZED, "Invalid token");
            return;
        }
        ctx.setSecurityContext(new BearerSecurityContext(principal,
            ctx.getSecurityContext().isSecure()));
    }

    private void abort(ContainerRequestContext ctx,
        Response.Status status, String message) {
        // TEXT_PLAIN error - HTTP status code carries the semantic meaning
        ctx.abortWith(Response.status(status)
            .entity("error=" + message + "\nstatus=" + status.getStatusCode()
+ "\n")
            .type(MediaType.TEXT_PLAIN)
            .header(HttpHeaders.WWW_AUTHENTICATE, "Bearer realm=\"api\")")
            .build());
    }
}

```

4.5 Custom SecurityContext Implementation

```

import javax.ws.rs.core.SecurityContext;

```

```

public class BearerSecurityContext implements SecurityContext {

    private final UserPrincipal principal;
    private final boolean secure;

    public BearerSecurityContext(UserPrincipal principal, boolean secure) {
        this.principal = principal;
        this.secure = secure;
    }

    @Override
    public java.security.Principal getUserPrincipal() { return principal; }

    @Override
    public boolean isUserInRole(String role) {
        return principal.getRoles().contains(role);
    }

    @Override public boolean isSecure() { return secure; }
    @Override public String getAuthenticationScheme() { return "Bearer"; }
}

public class UserPrincipal implements java.security.Principal {
    private final String name;
    private final Set<String> roles;
    public UserPrincipal(String name, Set<String> roles) {
        this.name = name; this.roles = roles;
    }
    @Override public String getName() { return name; }
    public Set<String> getRoles() { return roles; }
}

```

4.6 Declarative Authorisation — @RolesAllowed

Import role annotations from javax.annotation.security (NOT jakarta.annotation.security). Register RolesAllowedDynamicFeature in the Application class for Jersey to honour them:

```

// Register in the Application class:
// org.glassfish.jersey.server.filter.RolesAllowedDynamicFeature

import javax.annotation.security.DenyAll;
import javax.annotation.security.PermitAll;
import javax.annotation.security.RolesAllowed;
import javax.ws.rs.*;
import javax.ws.rs.core.*;

@Path("/admin")
@Produces(MediaType.TEXT_PLAIN) // responses are plain text
public class AdminResource {

    @GET @Path("users")
    @RolesAllowed("admin") // 403 if caller is not admin
    public String listUsers() {
        return "users=alice,bob\n";
    }

    @GET @Path("health")
    @PermitAll // no authentication required
    public Response health() {
        return Response.ok("status=UP\n").build();
    }
}

```

```

    }

    @DELETE @Path("system")
    @DenyAll // always 403, regardless of role
    public Response deleteSystem() {
        return Response.ok().build(); // never reached
    }

    @GET @Path("reports")
    @RolesAllowed({"admin", "manager"}) // either role allowed
    public String reports() {
        return "reports=Q1-2024,Q2-2024\n";
    }
}

```

4.7 @Secured Name-Binding for Selective Authentication

To apply authentication only to specific resources, use a custom @NameBinding annotation. Import from javax.ws.rs (NOT jakarta.ws.rs):

```

// 1. Define the binding annotation
import javax.ws.rs.NameBinding;
import java.lang.annotation.*;

@NameBinding
@Retention(RetentionPolicy.RUNTIME)
@Target({ElementType.TYPE, ElementType.METHOD})
public @interface Secured {}

// 2. Apply the annotation to the filter
@Provider
@Secured // filter only runs for @Secured endpoints
@Priority(Priorities.AUTHENTICATION)
public class BearerTokenFilter implements ContainerRequestFilter { ... }

// 3. Apply selectively to a resource method
@Path("/orders")
@Produces(MediaType.TEXT_PLAIN)
public class OrderResource {

    @GET // public – no @Secured, no authentication
    public String listPublic() {
        return "orders=ORD-001,ORD-002\n";
    }

    @POST
    @Secured // requires valid Bearer token
    @Consumes(MediaType.TEXT_PLAIN)
    public Response create(String order, @Context SecurityContext sc) {
        String user = sc.getUserPrincipal().getName();
        return Response.status(201)
            .entity("created=true\ncreatedBy=" + user + "\n")
            .build();
    }
}

```

4.8 Complete Security Flow

Request flow for POST /orders with Bearer token

1. Client sends: POST /orders Authorization: Bearer <token>
2. @Priority(AUTHENTICATION) BearerTokenFilter runs first
3. Filter extracts and validates the token → extracts username + roles
4. Filter calls ctx.setSecurityContext(new BearerSecurityContext(...))
5. JAX-RS dispatches to OrderResource.create()
6. @RolesAllowed("user") check: Jersey calls sc.isUserInRole("user")
7. Resource method calls sc.getUserPrincipal().getName() for audit logging
8. Response returned to client (TEXT_PLAIN body, appropriate status code)

4.9 Review Questions

10. What HTTP status code should an authentication filter return when the Authorization header is completely absent? What header should accompany it?
11. Explain the difference between @RolesAllowed and checking security.isUserInRole() programmatically. When would you choose each?
12. A resource class has @RolesAllowed("admin") at class level, and one method has @PermitAll. Which annotation wins for that method?
13. What is the purpose of @Priority(Priorities.AUTHENTICATION) on a ContainerRequestFilter? What happens if two filters have the same priority?

Appendix — Quick Reference

javax.* vs jakarta.* — Tomcat 9 Package Guide

Use case	Correct import	Do NOT use
JAX-RS core	<code>javax.ws.rs.*</code>	<code>jakarta.ws.rs.*</code>
JAX-RS container	<code>javax.ws.rs.container.*</code>	<code>jakarta.ws.rs.container.*</code>
JAX-RS extensions	<code>javax.ws.rs.ext.*</code>	<code>jakarta.ws.rs.ext.*</code>
Servlet API	<code>javax.servlet.*</code>	<code>jakarta.servlet.*</code>
HTTP Servlet	<code>javax.servlet.http.*</code>	<code>jakarta.servlet.http.*</code>
WebListener	<code>javax.servlet.annotation.*</code>	<code>jakarta.servlet.annotation.*</code>
Security annots	<code>javax.annotation.security.*</code>	<code>jakarta.annotation.security.*</code>
Priority	<code>javax.annotation.Priority</code>	<code>jakarta.annotation.Priority</code>
JNDI	<code>javax.naming.*</code>	(unchanged — no jakarta variant)
DataSource	<code>javax.sql.DataSource</code>	(unchanged — no jakarta variant)
JAXB API	<code>javax.xml.bind.*</code>	<code>jakarta.xml.bind.*</code>

Response Format Summary

Endpoint	Format	Reason
GET /health	APPLICATION_XML	JAXB @XmlRootElement bean
GET /config/*	TEXT_PLAIN	Config values are plain strings
GET /jndi/*	TEXT_PLAIN	JNDI lookup results are plain strings
GET /db/products	TEXT_PLAIN	JDBC tabular data — CSV rows
POST /db/products	TEXT_PLAIN body + response	Key=value input; status+id output
Security errors (4xx)	TEXT_PLAIN	HTTP status carries semantic meaning
@PermitAll health etc	TEXT_PLAIN	Consistent with module convention